

Lectura 7: A technical overview of Azure Cosmos DB

¿Cómo funciona el particionamiento horizontal en Cosmo DB?

Todos los datos dentro de un contenedor de Azure Cosmos DB se particionan horizontalmente y se administran mediante particiones de recursos. Una partición de recursos es un contenedor de datos particionado por una clave de partición especificada por el cliente. Es fundamental para escalabilidad y distribución. El sistema administra las particiones de forma transparente sin comprometer la disponibilidad, la consistencia, la latencia ni el rendimiento de un contenedor de Azure Cosmos DB.

Los clientes pueden escalar elásticamente el rendimiento de un contenedor con una granularidad de segundos o minutos. El escalado elástico del rendimiento mediante particiones horizontales de recursos requiere que cada partición de recursos sea capaz de proporcionar la parte del rendimiento total para un presupuesto determinado de recursos del sistema.

Comentario al modelo de consistencia en Cosmo DB

Azure CosmosDB permite a los desarrolladores elegir entre cinco modelos de consistencia bien definidos a lo largo del espectro de consistencia: fuerte, obsolescencia limitada, sesión, prefijo consistente y eventual. Esto se diferencia de las dos opciones limitadas que usualmente ofrecen otros servicios.

Los desarrolladores que usan Azure CosmosDB pueden configurar el nivel de consistencia predeterminado en su cuenta de base de datos. Para informar a los clientes sobre cualquier incumplimiento de los SLA de consistencia, emplean un verificador de linealización que opera continuamente sobre la telemetría del servicio.

Comente cómo los objetivos de diseño de Cosmo DB afectan la creación de aplicaciones

El objetivo era abordar los problemas fundamentales que enfrentan los desarrolladores que crean aplicaciones a escala de internet dentro de Microsoft.

- Permitir a los clientes escalar elásticamente el rendimiento y el almacenamiento según la demanda a nivel global. Esto permite que las aplicaciones tengan un buen rendimiento 99% del tiempo.

- Permitir a los clientes crear aplicaciones críticas y con alta capacidad de respuesta. Así las latencias de lectura y escritura en la aplicación son bajas.
- Garantizar que el sistema esté siempre activo. Esto ayuda a validar la disponibilidad de extremo a extremo de las aplicaciones.
- Permitir a los desarrolladores escribir aplicaciones correctas y distribuidas globalmente. El sistema debe ofrecer un modelo de programación intuitivo y predecible en torno a la consistencia de los datos, para poder solucionar errores y escalar la programación sin contratiempos.
- Ofrecer acuerdos de nivel de servicio (SLA) integrales y rigurosos con respaldo financiero. Así en caso de algún incumplimiento, el cliente tendrá el dinero perdido devuelto.
- Liberar a los desarrolladores de la carga de la gestión de esquemas/índices de bases de datos y el control de versiones. Así los desarrolladores no deben preocuparse de solucionar esta complejidad en aplicaciones.
- Compatibilidad nativa con múltiples modelos de datos y API populares para acceder a los datos. Esto facilita la creación de aplicaciones distribuidas.
- Operar a un coste muy bajo para trasladar los ahorros a los clientes, de manera que se evite que el costo de las aplicaciones sea astronómico.